

# Multi-Conversation Resume Chat

Give your chatbot a memory of its own conversations — build a ChatGPT-style sidebar that starts new chats and resumes old ones, entirely on the Streamlit frontend.

## WHAT THIS CHAPTER COVERS

- ▶ Why manual thread IDs break down
- ▶ Dynamic `thread_id` generation with `uuid4`
- ▶ Sidebar UI: New Chat + conversation list
- ▶ `reset_chat()` and the New Chat workflow
- ▶ Persisting thread history in session state
- ▶ `chatbot.get_state()` to fetch past messages
- ▶ Reformatting LangChain messages for the UI
- ▶ Resuming a past conversation on click

# Multi-Conversation Resume Chat

## RECAP — CHAPTER 12

Chapter 12 added token-by-token *streaming* to the chatbot using `chatbot.stream(..., stream_mode='messages')`, so responses appeared incrementally instead of all at once, greatly improving perceived responsiveness. The chatbot itself still used a single, hardcoded `thread_id` and an `InMemorySaver` checkpointer, meaning only one conversation could exist at a time and everything vanished on refresh.

## 1. The Problem: One Chatbot, One Conversation

### Definition

Up to Chapter 12, the chatbot supported exactly one active conversation at a time. Every call to `chatbot.stream(...)` passed a manually hardcoded `thread_id` such as "thread-1" inside the `config` dictionary. This worked only because there was ever a single, never-changing conversation to track.

### Why It Exists (as a limitation)

A real chat product — ChatGPT, Claude, or any assistant — lets a user start a brand-new conversation on a new topic while keeping every earlier conversation intact and revisitable. The existing chatbot could not do this: there was no mechanism to create additional threads, no list of past threads to choose from, and no way to reload the messages belonging to an older thread.

### Problem It Solves

This chapter introduces a *Resume Chat* feature: a sidebar with a **New Chat** button and a list of past conversations, so the user can freely switch between starting something new and continuing something old — exactly like a production chat application.

## IMPORTANT

This entire feature is built **only on the Streamlit frontend**. The LangGraph backend (`langgraph_backend.py`) is not touched at all — it already supports multiple independent threads because the checkpointer keys all state by `thread_id`. The limitation was purely in how the frontend was using it.

## 🔄 Quick Revision — The Problem

- The pre-Chapter-13 chatbot used one hardcoded `thread_id` for every message.
- There was no UI to start a new conversation or list past ones.
- The LangGraph backend already supports multiple threads via the checkpointer — nothing there needs to change.
- The fix is entirely a frontend/session-state problem, solved in Streamlit.

## 2. Breaking the Feature Into Tasks

Before writing any code, the large feature is broken into four smaller, independently testable tasks — a general engineering habit worth adopting for any non-trivial feature:

### 1. BUILD THE SIDEBAR UI

Add a title, a **New Chat** button, and a "My Conversations" section to the Streamlit sidebar.

### 2. GENERATE AND TRACK A DYNAMIC THREAD ID

Replace the hardcoded thread ID with a programmatically generated one, stored in session state.

### 3. WIRE UP THE NEW CHAT BUTTON

Clicking New Chat must generate a fresh thread ID, store it, and clear the visible message history.

### 4. PRESERVE AND RESUME PAST THREADS

Keep every thread ID ever created in a list, display them as clickable buttons, and reload a thread's full conversation when its button is clicked.

FRONTEND (Streamlit) — new session-state layer built this chapter

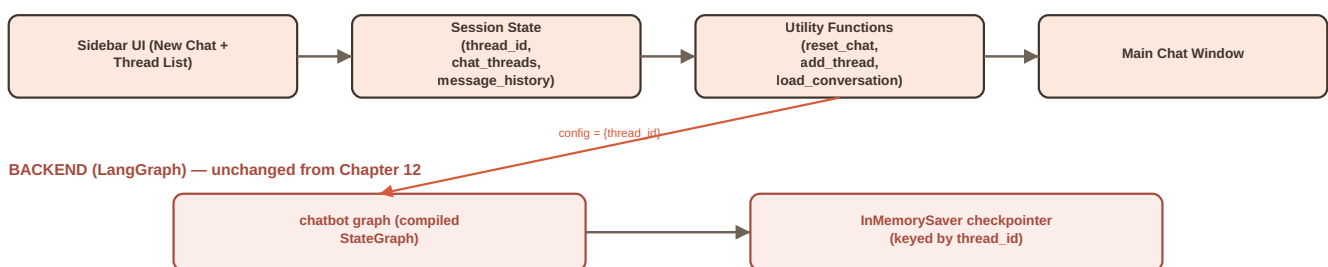


Figure 13.1 — Overall architecture — the new frontend session-state layer sits entirely in front of the unchanged LangGraph backend.

### 🔄 Quick Revision — Task Breakdown

- Task 1: sidebar UI (title, New Chat button, conversations header).
- Task 2: dynamic `thread_id` generation and storage in session state.
- Task 3: New Chat button resets `thread_id` and clears `message_history`.
- Task 4: a persistent list of all `thread_ids`, rendered as clickable resume buttons.

## 3. Task 1 — Sidebar UI

### How It Works

Streamlit renders anything called through `st.sidebar.*` in a collapsible left-hand panel. Three elements are added: a title, a button, and a header.

```
st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat()

st.sidebar.header('My Conversations')
```

At this point `reset_chat()` does not exist yet, so the button has no behaviour — it will be implemented in Task 3. Running the app now confirms only that the sidebar renders correctly and the existing chat still works.

### 🔄 Quick Revision — Sidebar UI

- `st.sidebar.title()`, `st.sidebar.button()`, and `st.sidebar.header()` build the sidebar layout.
- Anything called via `st.sidebar.*` renders in Streamlit's left panel, not the main chat area.
- The New Chat button is wired to a function (`reset_chat`) only in a later task.

## 4. Task 2 — Dynamic Thread ID Generation

### Why It Exists

A hardcoded `thread_id` like `"thread-1"` cannot represent an arbitrary, unknown number of conversations a user might create by clicking New Chat repeatedly. Thread IDs must instead be generated *programmatically*, guaranteed to be unique every time.

## How It Works

Python's built-in `uuid` module solves this. `uuid.uuid4()` generates a random, effectively-unique 128-bit identifier on every call — ideal as a thread key.

```
import uuid

# Utility Functions
def generate_thread_id():
    thread_id = uuid.uuid4()
    return thread_id
```

This new ID must then be created once (on first page load) and stored in Streamlit's `st.session_state`, which persists values across Streamlit's script reruns for the duration of a browser session:

```
# Session Setup
if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()
```

Finally, every place in the code that previously sent a hardcoded thread ID must instead read from session state:

### BEFORE (HARDCODED)

```
CONFIG = {
    'configurable': {
        'thread_id': 'thread-1'
    }
}
```

### AFTER (DYNAMIC)

```
CONFIG = {
    'configurable': {
        'thread_id':
            st.session_state['thread_id']
    }
}
```

### COMMON MISTAKE

Generating a new `thread_id` unconditionally on every script rerun (which Streamlit does on every interaction) would silently create a brand-new thread on every keystroke or button click, destroying continuity. The `if 'thread_id' not in st.session_state` guard is essential — it ensures the ID is generated once and then persists.

### 🔄 Quick Revision — Dynamic Thread ID

- `uuid.uuid4()` generates a random, effectively unique ID for each new conversation.
- `generate_thread_id()` wraps this call as a reusable utility function.
- The ID is stored in `st.session_state` so it survives Streamlit's automatic script reruns.
- Every hardcoded `thread_id` in `CONFIG` must be replaced with `st.session_state['thread_id']`.

## 5. Task 3 — Wiring the New Chat Button

### Problem It Solves

Clicking New Chat must start a genuinely fresh conversation: a new thread ID, and an empty chat window. Three things need to happen together, so they are grouped into one utility function, `reset_chat()`.

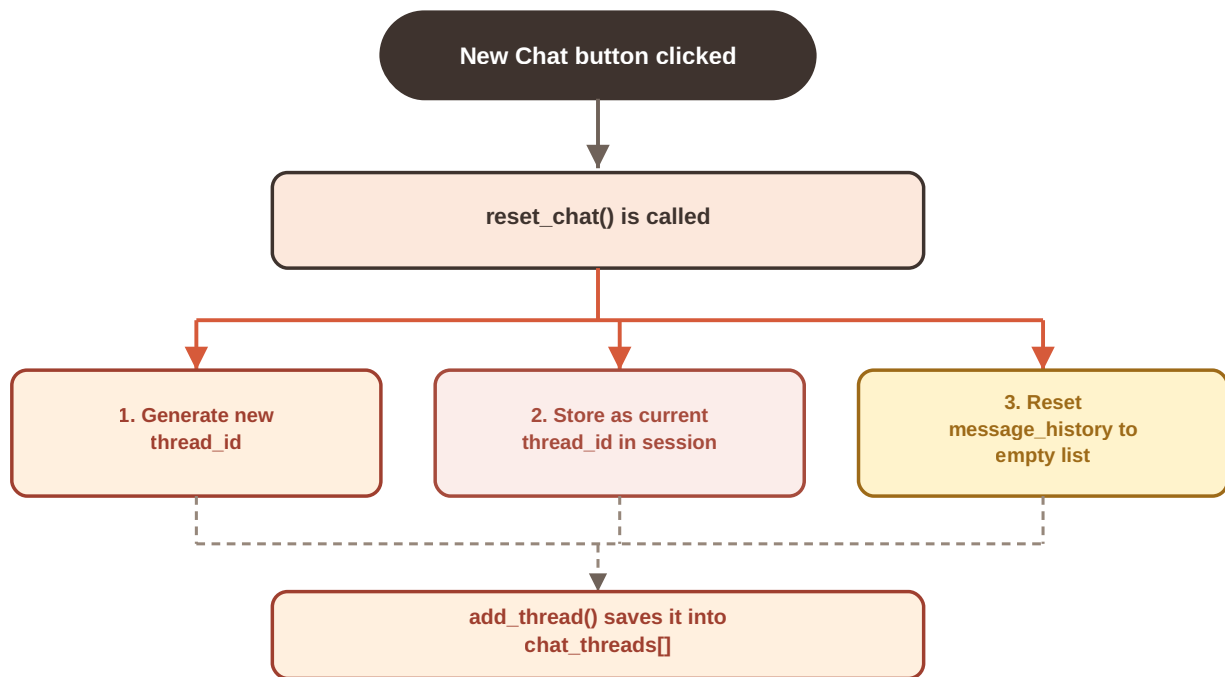


Figure 13.2 — What happens, in order, when the New Chat button is clicked.

```
def reset_chat():
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    st.session_state['message_history'] = []
```

And the sidebar button is connected to it:

```
if st.sidebar.button('New Chat'):
    reset_chat()
```

Testing at this point confirms the visible chat window clears and a brand-new thread ID takes over — but the *previous* thread ID and its messages are now unreachable, since nothing has recorded that this older thread ever existed. This is addressed in Task 4.

### BEST PRACTICE

Group multi-step UI actions (generate ID, update session, clear messages) into a single named utility function such as `reset_chat()` rather than inlining the steps at the button call site. It keeps the sidebar wiring readable and makes the action reusable and testable.

#### 🔄 Quick Revision — New Chat Wiring

- `reset_chat()` generates a new `thread_id`, stores it, and empties `message_history`.
- The New Chat button simply calls `reset_chat()` when clicked.
- At this stage, older threads are cleared from view but not yet remembered anywhere.

## 6. Task 4 — Preserving and Resuming Past Threads

### Why It Exists

Session state currently tracks only the *current* thread ID. To resume old conversations, every thread ID ever created in this session must be retained in a list, independent of which one is currently active.

### How It Works — Step 1: A Persistent List of Threads

A new session-state list, `chat_threads`, stores every thread ID that has ever existed:

```
if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = []
```

A thread needs to be appended to this list exactly twice: once on initial page load (for the very first thread) and once every time `reset_chat()` runs (for each new conversation). Both call sites reuse one utility function, `add_thread()`, which avoids duplicate entries:

```
def add_thread(thread_id):
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)
```

This is called once right after session setup, and again inside `reset_chat()`:

```
# after session setup, runs on every script load
add_thread(st.session_state['thread_id'])

def reset_chat():
    thread_id = generate_thread_id()
    add_thread(thread_id)
    st.session_state['thread_id'] = thread_id
    st.session_state['message_history'] = []
```

#### INTERVIEW TIP

If asked why `add_thread()` checks membership before appending, explain that Streamlit reruns the entire script on every interaction — without the guard, the same thread ID could be appended repeatedly every time the script reruns, corrupting the sidebar list with duplicates.

#### 🔄 Quick Revision — Persisting Threads

- `chat_threads` is a session-state list holding every `thread_id` created so far.
- `add_thread(thread_id)` appends only if the ID is not already present, preventing duplicates.
- `add_thread()` is called once after session setup and once inside `reset_chat()`.

## 7. Displaying and Selecting a Thread

### How It Works

The sidebar loops over `chat_threads` and renders each one as a clickable button (not plain text), so each thread can be selected:

```
for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(f'{thread_id}'):
        st.session_state['thread_id'] = thread_id
        # ... load and display this thread's messages ...
```

The slice `[::-1]` reverses the list so the most recently created thread appears at the top of the sidebar, matching the familiar convention from tools like ChatGPT.

#### REAL WORLD INSIGHT

Production chat apps go further and give each thread a human-readable title (often auto-generated from the first user message) instead of a raw UUID. This chapter intentionally keeps it simple and displays the UUID directly — giving threads friendly names is left as a self-directed exercise.

### 🔄 Quick Revision — Thread List Display

- Each `thread_id` in `chat_threads` is rendered as its own sidebar button.
- Reversing the list with `[::-1]` shows the newest conversation first.
- Clicking a thread button updates `st.session_state['thread_id']` to that thread.

## 8. Loading a Past Conversation's Messages

### Definition

Selecting a thread updates which thread is *active*, but the visible `message_history` must also be repopulated with that thread's actual past messages before anything shows on screen.

### How It Works

LangGraph's checkpointer already stores every message per thread. `chatbot.get_state(config)` retrieves a `StateSnapshot` object for a given thread; its `.values` attribute is the state dictionary, and `.values['messages']` is the full list of LangChain message objects for that thread:

```
def load_conversation(thread_id):
    return chatbot.get_state(
        config = {'configurable': {'thread_id': thread_id}}
    ).values['messages']
```

#### RESUME-CHAT SEQUENCE



Figure 13.3 — Full resume-chat sequence, from clicking a thread button to a re-rendered chat window.

### 🔄 Quick Revision — Fetching Past Messages

- `chatbot.get_state(config)` returns a `StateSnapshot` for a given `thread_id`.
- `.values` is the underlying state dict; `.values['messages']` is the message list.
- `load_conversation(thread_id)` wraps this into a single reusable utility function.

## 9. Reconciling the Message Format

### Problem It Solves

`load_conversation()` returns LangChain message objects (`HumanMessage`, `AIMessage`), but `st.session_state['message_history']` stores plain dictionaries shaped like `{'role': ..., 'content': ...}` for rendering with `st.chat_message()`. These two formats are incompatible and must be converted.

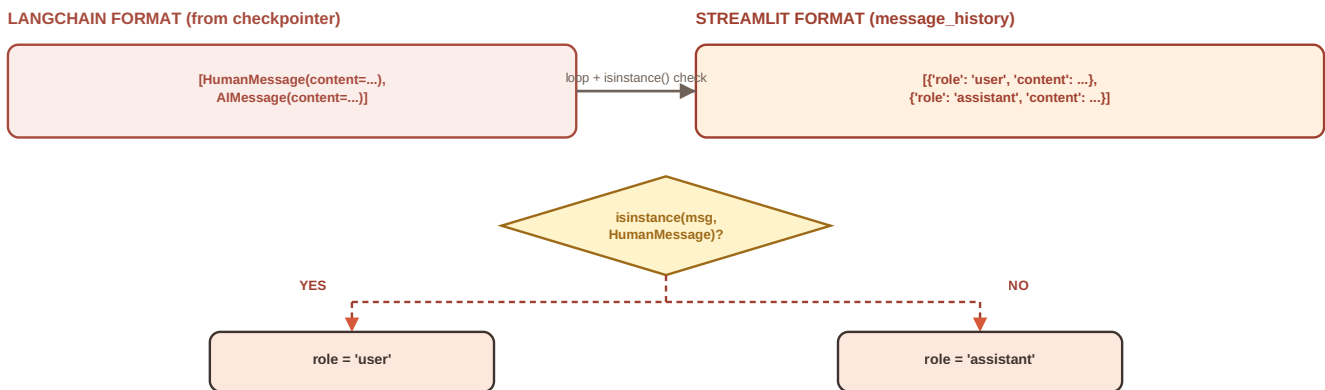


Figure 13.4 — Converting LangChain message objects into the `{role, content}` dict format used by the chat UI.

```
temp_messages = []

for message in messages:
    if isinstance(message, HumanMessage):
        role = 'user'
    else:
        role = 'assistant'
    temp_messages.append({'role': role, 'content': message.content})

st.session_state['message_history'] = temp_messages
```

`isinstance(message, HumanMessage)` distinguishes user messages from AI messages (everything else in this simple two-role chatbot is treated as an `AIMessage`). The resulting `temp_messages` list fully replaces `message_history`, and the existing rendering loop (already written for the chat window) displays it without any further changes.

### COMMON MISTAKE

When a thread button is clicked, it is easy to forget to also update `st.session_state['thread_id']` to the clicked thread. Without this, subsequent messages typed by the user would be appended to the *previous* thread in LangGraph, even though the UI is visually showing a different, older conversation.

```

for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(f'{thread_id}'):
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)

        temp_messages = []
        for message in messages:
            if isinstance(message, HumanMessage):
                role = 'user'
            else:
                role = 'assistant'
            temp_messages.append({'role': role, 'content': message.content})

        st.session_state['message_history'] = temp_messages

```

### 🔄 Quick Revision — Format Reconciliation

- LangChain returns HumanMessage/AIMessage objects; the UI needs {'role', 'content'} dicts.
- isinstance(message, HumanMessage) maps a message to role 'user', else 'assistant'.
- Clicking a thread button must update BOTH the active thread\_id AND message\_history.
- The existing chat-rendering loop needs no changes once message\_history is in the right shape.

## 10. Complete Frontend Reference

The full, final frontend for this chapter (backend unchanged from Chapter 12) is reproduced below, exactly as implemented in `streamlit_frontend_threading.py`:

```

import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage
import uuid

# ***** Utility Functions *****

def generate_thread_id():
    thread_id = uuid.uuid4()
    return thread_id

def reset_chat():
    thread_id = generate_thread_id()
    add_thread(thread_id)
    st.session_state['thread_id'] = thread_id
    st.session_state['message_history'] = []

def add_thread(thread_id):
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)

def load_conversation(thread_id):
    return chatbot.get_state(
        config = {'configurable': {'thread_id': thread_id}}
    ).values['messages']

```

```

# ***** Session Setup *****

if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = []

add_thread(st.session_state['thread_id'])

# ***** Sidebar UI *****

st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat()

st.sidebar.header('My Conversations')

```

```

for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(f'{thread_id}'):
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)

        temp_messages = []
        for message in messages:
            if isinstance(message, HumanMessage):
                role = 'user'
            else:
                role = 'assistant'
            temp_messages.append({'role': role, 'content': message.content})

        st.session_state['message_history'] = temp_messages

```

```

# ***** Main UI *****
# (unchanged from Chapter 12 – renders message_history, then
# streams a new response and appends both turns to message_history)

for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

user_input = st.chat_input('Type here')

if user_input:
    st.session_state['message_history'].append({'role': 'user', 'content': user_input})
    CONFIG = {'configurable': {'thread_id': st.session_state['thread_id']}}

    with st.chat_message('user'):
        st.text(user_input)

    with st.chat_message('assistant'):
        ai_message = st.write_stream(
            message_chunk.content for message_chunk, metadata in chatbot.stream(
                {'messages': [HumanMessage(content = user_input)]},
                config = CONFIG,
                stream_mode = 'messages'
            )
        )

    st.session_state['message_history'].append({'role': 'assistant', 'content': ai_message})

```

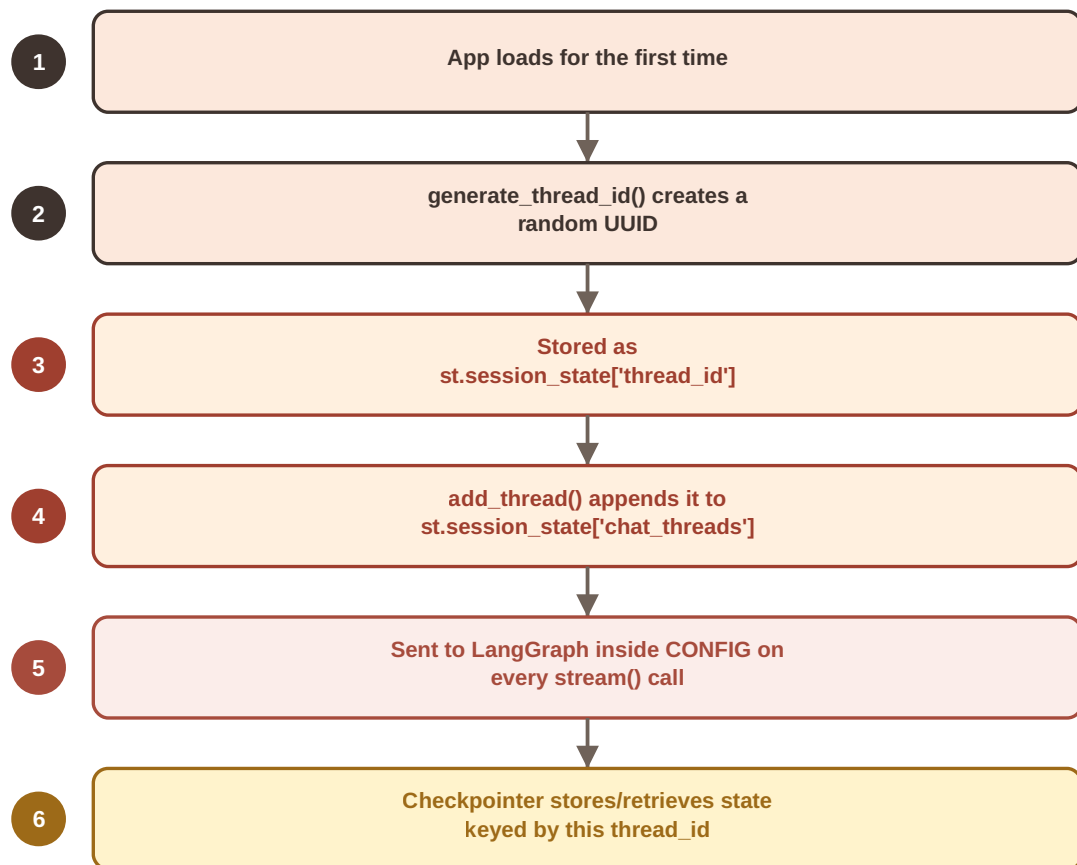


Figure 13.5 — How a fresh `thread_id` flows through the app, from generation to checkpointer lookup.

### 🔄 Quick Revision — Full Implementation

- The LangGraph backend file (`langgraph_backend.py`) required zero changes for this feature.
- All new logic lives in the frontend: three utility functions plus sidebar wiring.
- The Main UI rendering/streaming logic from Chapter 12 is reused completely unmodified.

## K. Key Takeaways

- **Root cause:** a single hardcoded `thread_id` meant the pre-Chapter-13 chatbot could only ever hold one conversation.
- **Dynamic thread IDs:** `uuid.uuid4()` generates a fresh, unique `thread_id` per conversation, stored in `st.session_state`.
- **reset\_chat():** the New Chat action bundles generating a new `thread_id`, saving it, and clearing `message_history`.
- **chat\_threads list:** a session-state list retains every `thread_id` ever created, populated via `add_thread()` to avoid duplicates.

- **Sidebar thread buttons:** each stored `thread_id` is rendered as a clickable button, newest first via `[::-1]`.
- **`chatbot.get_state()`:** retrieves a `StateSnapshot` for any `thread_id`; `.values['messages']` holds its full message list.
- **Format reconciliation:** LangChain message objects are converted to `{'role', 'content'}` dicts via `isinstance(message, HumanMessage)` checks before being placed in `message_history`.
- **Backend untouched:** the entire feature is implemented on the Streamlit frontend; the LangGraph graph and checkpointers required no changes.

## R. Revision Sheet

### **generate\_thread\_id()**

Utility function that returns `uuid.uuid4()` — a fresh, unique thread identifier for a new conversation.

### **reset\_chat()**

Generates a new `thread_id`, registers it via `add_thread()`, sets it as the active thread, and empties `message_history`.

### **add\_thread(thread\_id)**

Appends a `thread_id` to `st.session_state['chat_threads']` only if it isn't already present, preventing duplicate sidebar entries.

### **load\_conversation(thread\_id)**

Calls `chatbot.get_state(config).values['messages']` to fetch all LangChain messages stored for a given thread.

### **chat\_threads**

A session-state list of every `thread_id` created during the session; iterated in reverse to show newest conversations first.

### **Format conversion**

`isinstance(message, HumanMessage)` maps LangChain messages to 'user'/'assistant' roles before storing as `{'role', 'content'}` dicts.

## I. Interview Questions

**Q1.** Why can't a chat application rely on a single hardcoded `thread_id` to support multiple conversations?

**Q2.** What does `uuid.uuid4()` guarantee, and why is it suitable for generating thread identifiers?

**Q3.** Why must new-thread-ID generation be guarded with an "if not in session\_state" check rather than run unconditionally on every script rerun?

**Q4.** What three things does `reset_chat()` need to do, and in what order, for the New Chat button to work correctly?

**Q5.** Why does `add_thread()` check for existing membership before appending a `thread_id` to `chat_threads`?

**Q6.** What does `chatbot.get_state(config)` return, and how do you extract the message list from it?

**Q7.** Why are LangChain message objects incompatible with Streamlit's `message_history` format, and how is the conversion performed?

**Q8.** What would go wrong if, when resuming a thread, the code updated `message_history` but forgot to update `st.session_state['thread_id']`?

**Q9.** Why did this entire feature require no changes to the LangGraph backend (`langgraph_backend.py`)?

**Q10.** How would you modify this design so that each conversation shows a human-readable title instead of a raw UUID?

## F. Further Reading

- LangGraph Persistence documentation — checkpoints and thread-scoped state ([langchain-ai.github.io/langgraph](https://langchain-ai.github.io/langgraph))
- Streamlit Session State documentation — `st.session_state` API reference ([docs.streamlit.io](https://docs.streamlit.io))
- Python uuid module documentation — `uuid1/uuid4` and identifier generation ([docs.python.org](https://docs.python.org))
- Streamlit Sidebar API reference — `st.sidebar` elements ([docs.streamlit.io](https://docs.streamlit.io))